Latest updates: https://dl.acm.org/doi/10.1145/3061639.3062283

RESEARCH-ARTICLE

Minimizing Cluster Number with Clip Shifting in Hotspot Pattern Classification

KUAN-JUNG CHEN, National Taiwan University of Science and Technology, Taipei, Taiwan

YU-KAI CHUANG, National Taiwan University of Science and Technology, Taipei, Taiwan

BO-YI YU, National Taiwan University of Science and Technology, Taipei, Taiwan

SHAO-YUN FANG, National Taiwan University of Science and Technology, Taipei, Taiwan

Open Access Support provided by:

National Taiwan University of Science and Technology

PDF Download
3061639.3062283.pdf
04 April 2026
Total Citations: 18
Total Downloads: 298

Published: 18 June 2017

Citation in BibTeX format

DAC '17: The 54th Annual Design Automation Conference 2017
June 18 - 22, 2017
TX, Austin, USA

Conference Sponsors:
SIGDA

Minimizing Cluster Number with Clip Shifting in Hotspot Pattern Classification*

Kuan-Jung Chen, Yu-Kai Chuang, Bo-Yi Yu, and Shao-Yun Fang
Department of Electrical Engineering, National Taiwan University of Science and Technology, Taipei 106, Taiwan
{m10407412, m10507410, m10507413, syfang}@mail.ntust.edu.tw

ABSTRACT

With the rapid advance of semiconductor process technologies, layout features in integrated circuits (ICs) become highly prone to process variations. Lithography hotspots are a set of problematic layout patterns with poor printability even if they pass design rule checking (DRC). These hotspots need to be detected and fixed as early as possible in the design flow to improve manufacturability and yield. While most of existing studies focus on hotspot detection, hotspot pattern classification is rarely addressed but plays an important role in determining the efficiency of hotspot detection. In hotspot pattern classification, similar hotspots are classified into a cluster with tolerance constraints, and a representative hotspot is chosen for each cluster for future application. To minimize the problem size of subsequent hotspot detection, the number of clusters, and thus the number of representative hotspots, should be minimized. In this paper, a clip shifting method is adopted to further reduce the cluster number during hotspot classification. We propose a two-stage pattern matching algorithm flow and derive optimal solutions by solving a set cover problem. Experiment results show that our flow can reduce the cluster number by about 30% compared to reference results in 2016 CAD Contest at ICCAD.

CCS CONCEPTS

• **Hardware** → **Electronic design automation**; *Design for manufacturability*;

KEYWORDS

Hotspot pattern classification, clip shifting, cluster number minimization

1 INTRODUCTION

With the rapid advance of semiconductor process technologies, layout features in integrated circuits (ICs) become more and more prone to lithography hotspots because feature sizes become much smaller than the lithographic wavelength. Lithography hotspots are a set of problematic layout patterns with poor printability even if they pass design rule checking (DRC). These hotspots need to be detected and fixed as early as possible in the design flow to improve manufacturability and yield.

In traditional design flow, lithography simulations are used to identify hotspot patterns [4, 5]. Despite the high accuracy, simulation-based approaches suffer from the extremely high computational complexity, especially on a full-chip scale. To avoid unacceptable long computation time, geometry-based hotspot detection algorithms are proposed in recent years, which require proper description of geometric characteristics for hotspot patterns. In general, geometry-based methods can be classified into three categories: 1) pattern matching, 2) machine learning, and 3) hybrid method. Fig. 1 shows the flow of geometry-based hotspot detection methods. In the pattern classification phase, pattern matching-based methods collect a set of known hotspot patterns while the machine learning-based methods collect both hotspot and non-hotspot patterns. After the pattern classification, the representative patterns of clusters are directly used for pattern matching-based hotspot detection. On the other hand, machine learning-based methods use these representative patterns to train a hotspot classification model and then use the model for hotspot detection.

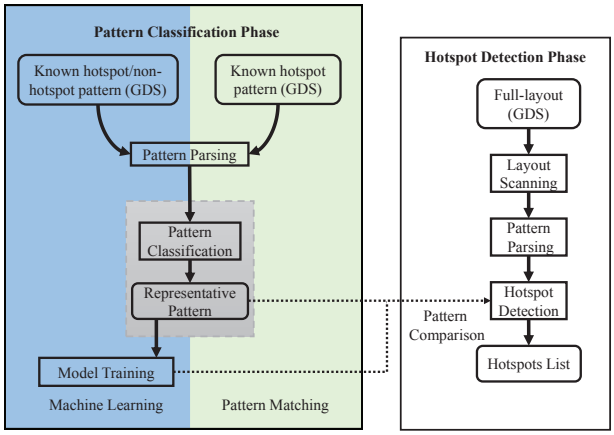

Figure 1: Geometry hotspot detection flow

Pattern matching-based methods have the advantage of short runtime and full detection capability on a set of pre-characterized patterns but cannot handle unknown hotspots [7, 9–11]. Machine learning-based methods usually employ artificial neural network (ANN) or support vector machine (SVM) models to predict fuzzy hotspot patterns but suffer from lower detection accuracy and higher false alarm rates [1, 2]. The hybrid-based approaches try to use both pattern matching and machine learning techniques to benefit from the both advantages [6, 8, 12].

No matter which method is applied to hotspot detection, the detection accuracy and efficiency have a high dependency on pattern classification technique. Therefore, finding a minimized number of hotspot patterns which are representative of problematic features is the major objective in pattern classification. In [10], they use a library of range patterns which is provided by the fab to apply an intuitive classification for Range Pattern Matching (RPM). A scoring mechanism is proposed to differentiate the various patterns based on the DRC rules and identify patterns which have the greater probabilities to be problematic patterns. In [7], the proposed pattern classification approach computes the geometry similarities between different clips by calculating the pattern overlapping areas. Yet, after considering the shape variation of metal features, this distance metric becomes very sensitive, leading to worse classification accuracy. To address the above concerns, an improved tangent space-based distance metric is adopted in [3]. This metric can handle multiple patterns and tolerate small variations on pattern edges but has higher computational cost.

*This work was partially supported by MOST 104-2628-E-011-009-MY3.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC '17, June 18-22, 2017, Austin, TX, USA
© 2017 ACM. ISBN 978-1-4503-4927-7/17/06. . . $15.00
DOI: http://dx.doi.org/10.1145/3061639.3062283

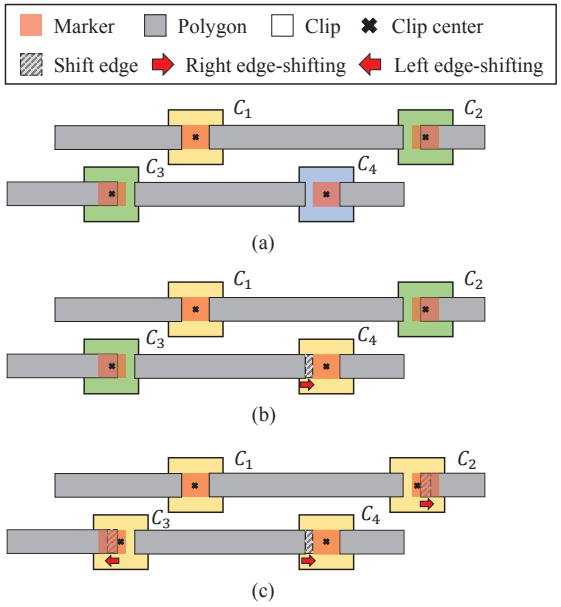

Figure 2: An example of clip shifting to merge more clusters. (a) Four clips in three different clusters. (b) C_1 and C_4 are merged into a cluster with edge tolerance. (c) Merge all clips into one cluster with clip shifting.

The aforementioned pattern classification works only consider the tolerance of pattern shape variations. These variations are set to be very small to keep their original topologies, and thus the opportunity of pattern clustering is limited. It can be observed in Fig. 2 that by adopting slight pattern shifting, much more similar patterns can be clustered. In Fig. 2, a marker indicates where a hotspot is and a clip circles a range of layout pattern that causes the hotspot. The formal definitions of these terminologies will be defined in Section 2. Clips in the same color denote that they are in the same cluster. It can be observed that we have three clusters in Fig. 2(a), where C_2 and C_3 are actually the same pattern since they are mirror symmetric. By considering edge tolerance, C_1 and C_4 are considered to be similar hotspots and thus clustered, as shown in Fig. 2(b). By further applying clip shifting to C_2 and C_3, as shown in Fig. 2(c), all the clips are clustered together. For the cluster composed of these four clips, we can use C_1 as the representative clip because C_2, C_3, and C_4 can become C_1 with edge tolerance.

In this paper, we propose a novel hotspot pattern classification framework. To the best of our knowledge, it is the first work considering clip shifting to minimize the number of clusters. Our approach allows us to perform fast overall layout scanning to find all hotspot markers and the corresponding clips. Considering the pattern edge or area tolerance, our framework classifies the clips with similar geometry into a cluster. To further reduce the cluster number with clip shifting, a systematic approach is proposed to generate more useful candidate clips. Finally, we transform the problem into a set cover problem to simultaneously derive the optimal cluster number and determine a representative clip for each cluster. Experimental results show that without clip shifting, our flow can derive fewer cluster numbers on average compared to the reference results in 2016 CAD Contest at ICCAD. By adopting clip shifting, our flow can further reduce cluster numbers by about 30%.

The rest of this paper is organized as follows: Section 2 gives some terminologies and the problem formulation. In Section 3, we detail each step in the algorithm flow for hotspot pattern classification. Experimental results are presented in Section 4. Finally, we conclude our work in Section 5.

2 PROBLEM FORMULATION

Before giving the problem formulation, we first define some terminologies as follows:

Definition 1. Clip*: a clip is composed of a set of polygons in a rectangular or squared window extracted from the original layout as shown in Fig. 2(a). The clip size is given as an (w, h) pair, where w and h are respectively the width and the height of the window.*

Definition 2. Marker*: a polygonal marker indicates where a hotspot is and where a clip should be centered on the input layout. The center point of a clip has to fall within or on the edges of a marker polygon. Markers do not necessarily intersect with any layout pattern; that is, they can also be placed on the space among features. For example, the marker of C_1 in Fig. 2(a) do not intersect with any feature. Marker sizes and shapes could vary from clip to clip, but generally they are small compared to the clip size.*

Definition 3. Cluster*: a cluster is a group of clips that are similar to each other.*

Definition 4. Representative Clip*: a representative clip is a clip in a cluster that guarantees certain degree of similarity between itself and any other clip in the cluster. For example, C_1 in Fig .2(c) can be the representative clip since the others can be matched to C_1 with edge tolerance.*

To define the required degree of similarity between a representative clip and other clips in a cluster, two criteria are adopted as follows:

(1) *Area-constrained clustering (ACC)*: ACC limits the area difference between a representative clip and any other clip in the same cluster. The exclusive OR (XOR) operation is applied to define the area difference between two clips, as shown in Fig. 3. The ACC constraint is given as follows:

$$\frac{Area(XOR(R, C))}{w \times h} \leq (1 - a), \tag{1}$$

where $Area(XOR(R, C))$ is the area difference between a representative clip R and any clip C in the same cluster, $w \times h$ is the clip area, and a is the area matching constraint parameter indicating the least area match ratio that should be achieved between two clips. a is a real non-negative number no larger than 1, and it is usually set to be close to 1 in practical application. Setting a to be 1 indicates an exact area matching. A clip satisfying Equation (1) with a representative clip indicates that they can (but may not) be clustered. In contrast, if Equation (1) is not satisfied, R and C cannot be in the same cluster.

(2) *Edge-constrained clustering (ECC)*: ECC gives the tolerance that a pattern edge of a clip can be shifted by a limited amount compared to the corresponding edge of the representative clip in the same cluster. The limited shifting amount is given by e, which indicates how much an edge can be shifted either inward or outward. Multiple edges can simultaneously shift with different amounts no larger than e. Fig. 4(a) illustrates an example clip. By shifting several edges with limited amounts, as shown in Fig. 4(b), the representative clip becomes another clip shown in Fig. 4(c), and thus clips in Figs. 4(a) and (c) can be in the same cluster. The parameter e is a non-negative real number and is generally set to be close to 0 in practical application.

Having the above terminologies, the problem formulation is given as follows:

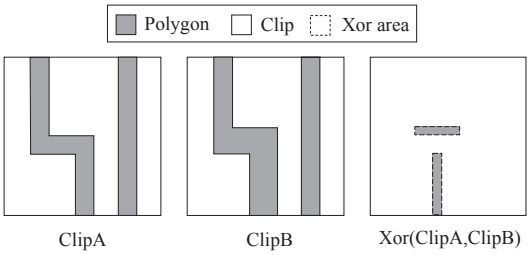

Figure 3: The area difference between two clips with the XOR operation.

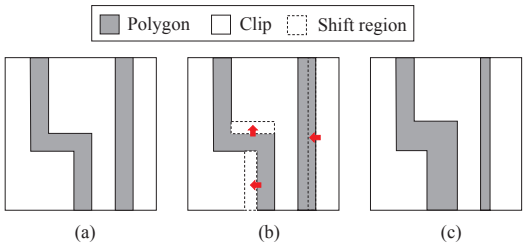

Figure 4: An example of ECC. (a) A representative clip. (b) Multiple edge shiftings satisfying the ECC constraint. (c) A clip that is able to be clustered with the representative clip in (a).

PROBLEM 1. *Given a layout with a set of hotspot markers, clip dimensions, area constraint parameter a, and edge constraint parameter e, cover each marker with a clip and classify the clips into clusters such that the number of clusters is minimized. The clips in each cluster have to follow either ACC or ECC, as specified by the user, and each clip center has to fall within or on the edges of the corresponding marker.*

3 HOTSPOT PATTERN CLASSIFICATION

In this section, we introduce the proposed pattern classification algorithms. The overall flow is presented in Section 3.1. Then, we detail the major steps in the following subsections.

3.1 Pattern Classification Flow

Fig. 5 shows the overall flow of our pattern classification framework. Given an input layout and a set of markers, we first partition the layout into grids according to the clip size and record which polygonal layout features are overlapped with each grid. This pre-processing operation will greatly speed up the following steps as the grid data structure contributes to quick access of local layout information. Then, a clip is created for each marker, and we set the center of the clip to be overlapped with the center of the marker by default. After that, we divide our pattern classification flow into two stages, clip exact matching and area/edge-tolerant clip matching considering clip shifting. The former detects clips that are exactly the same and regards them as a single clip to greatly reduce the problem size, and the latter uses sophisticated algorithms to explore which clips could be further clustered by considering ACC/ECC and clip shifting. Finally, the clip clustering problem is transformed into a set cover problem and can be either efficiently solved with a greedy algorithm or optimally solved with an ILP formulation. Each step for pattern classification is detailed in the following.

3.2 Exact Clip Matching

To efficiently determine whether two clips are exactly identical, a row-based tile data structure is adopted to represent each clip.

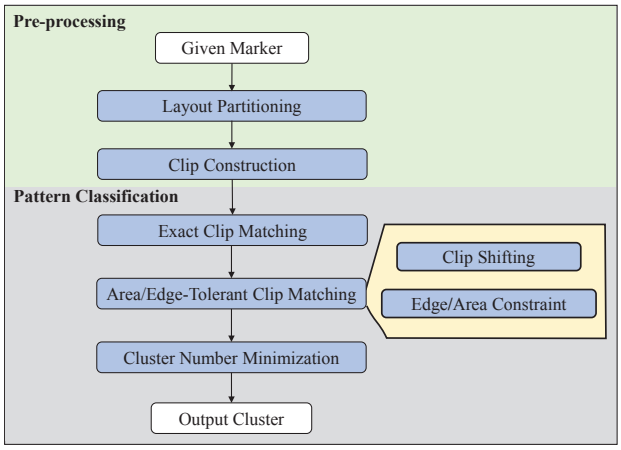

Figure 5: Our pattern classification flow.

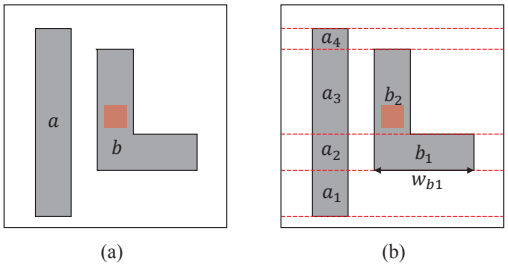

Figure 6: An example of creating row-based clip data structure. (a) A given clip. (b) A clip is sliced into rows according to the horizontal pattern edges

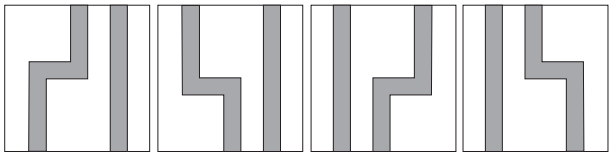

Figure 7: Four mirror symmetric clips.

As shown in Fig. 6, a clip is sliced into rows according to the horizontal pattern edges within the clip. For each row, tiles can be classified into two categories, solid (gray) tiles and space (white) tiles, which are respectively covered and uncovered by layout features. A pair of parameters $(type_t, width_t)$ is used for a tile t, where $type_t$ is either solid or space to indicate the tile type, and $width_t$ records the width of t. For example, tile b_1 is recorded as $(solid, w_{b1})$. All the tile parameters of a clip are stored in a two dimensional linked list. Thus, by comparing the linked lists of two clips, it can be easily checked whether they are identical or not. To speed up the matching process, quick comparison can be first made on the row numbers, the tile numbers and the row heights to filter out impossible matchings and avoid the linear-time scanning process. Note that mirror symmetric patterns are regarded as identical and thus are merged in this stage. Fig. 7 shows four mirror symmetric patterns. These patterns can also be simply detected by using the linked list data structure.

We first expand the boundary of each clip such that the clip size becomes $(w_c + w_m) \times (h_c + h_m)$, where w_c, h_c, w_m, and h_m are separate the widths and the heights of a clip and a marker. As

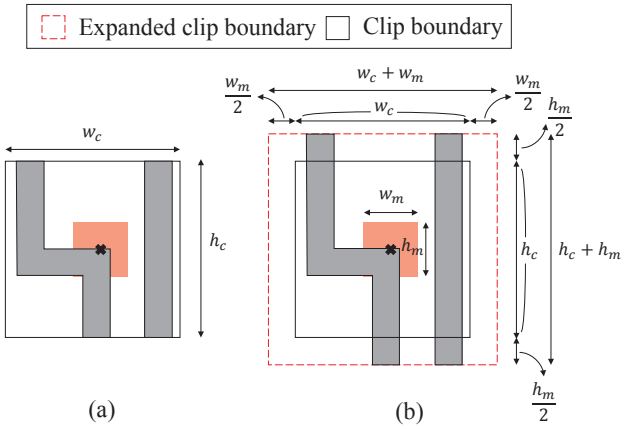

Figure 8: An example for expanded clip. (a) A given clip. (b) An expanded clip.

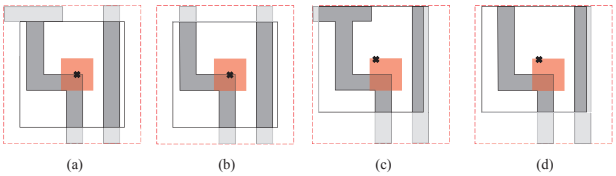

Figure 9: An example for two clips may be different if their expanded clips are not identical. (a)(b) Two given clips with center point on the marker center. (c)(d) Two given clips shifted center point to left-up corner.

illustrated in Fig. 8, an original clip is given in Fig. 8(a), and its expanded one is shown in Fig. 8(b). The expanded clip boundary of a marker gives the possible clip placement range if clip shifting is considered. Therefore, if the expanded clips of two markers are verified to be exactly the same by using the above comparison method, they can be directly clustered. In contrast, if the layout patterns of two clips are identical in the original clip boundaries but are different in the expanded clip boundaries, the two clips are very likely to become non-clusterable if their clip centers are no more placed on the marker centers. In this circumstance, we do not cluster them in this stage. For example, Figs. 9(a) and (b) are the identical clips with their center points on the marker center. However, they become different and non-clusterable when the two clips shift to the left-up corner, as shown in Figs. 9(c) and (d).

Note that after clustering identical clips using exact clip matching, every clip in each cluster can serve as the representative clip because all the clips in a cluster are exactly the same. Empirically, the cluster number after this stage can be greatly reduced compared to the marker number, which greatly reduces the problem size of the later clustering problem considering clip shifting.

3.3 Area/Edge-Tolerant Clip Matching with Clip Shifting

Two non-clusterable clusters from exact clip matching might become clusterable if clip shifting is adopted (an example will be presented in Section 3.4). A clip can be shifted as long as the clip center falls within its marker. However, enumerating all shifting possibilities for each clip would result in intractably huge problem sizes and thus is impractical. How to generate clip candidates which can effectively contribute to further cluster merging becomes the major challenge of this problem.

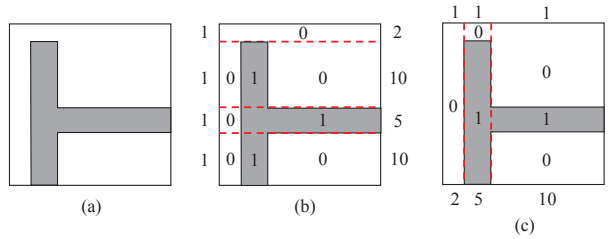

Figure 10: String-based encoding. (a) A given clip. (b) The string encoding of the right side. (c) The string encoding of the bottom side.

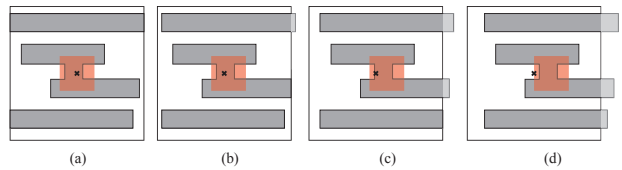

Figure 11: An example process for clip candidate generation by leftward shifting. (a) A given clip. (b)(c)(d) Clip candidates are generated when pattern edges touch the clip boundary.

3.3.1 Clip Candidate Generation with Clip Shifting. We propose a clip candidate generation approach considering clip shifting, which was inspired by a string-based layout classification approach used for hotspot detection [12]. Fig. 10 shows the string-based encoding for a layout clip, where a space tile is encoded as '0' and a solid tile is encoded as '1'. A binary string sequence can be generated for each side of a clip by adding an additional '1' to the opposite side, and then each binary string is converted into a decimal number. As illustrated in Fig. 10(b), to generate the binary string sequence of the right side, an additional '1' is added on the left side for each row, and the binary string sequence $< 10_2, 1010_2, 101_2, 1010_2 >$ is converted into $< 2, 10, 5, 10 >$. Fig. 10(c) shows the sequence generation for the bottom side of the clip. Then, the topology of a clip can be described by the string encoding of the four sides. The similarity between two string encodings is highly correlated with the similarity between the two clips.

To generate useful and significant clip candidates, candidates that contain different pattern topologies and thus different string encodings are desirable. It can be observed that the string encoding of a clip changes if a pattern edge touches the clip boundary during clip shifting. Fig. 11 illustrates an example. Fig. 11(a) is a representative clip of a cluster centered on the marker center. By shifting the clip to the left until the rightmost pattern edge touches the clip boundary, a new clip candidate with different string encoding can be generated, as shown in Fig. 11(b). Fig. 11(c) shows another generated clip candidate by further shifting the representative clip to the left. As the clip center reaches the marker boundary, the clip is also regarded as a candidate, as the one in Fig. 11(d). This is because based on the empirical observations, this kind of clip candidates will be very useful for cluster number reduction. Note that in our work, we only consider clip shifting in a single direction (left, right, up, and down). Simultaneously considering more than one direction can generate more candidate clips with the trade-off larger problem sizes of the later cluster number minimization problem.

3.3.2 Area/Edge-Tolerant Clip Matching. Having all the candidate clips including the representative clips of current clusters from exact clip matching and the generated clips with clip shifting, area/edge-tolerant clip matching is performed to check which

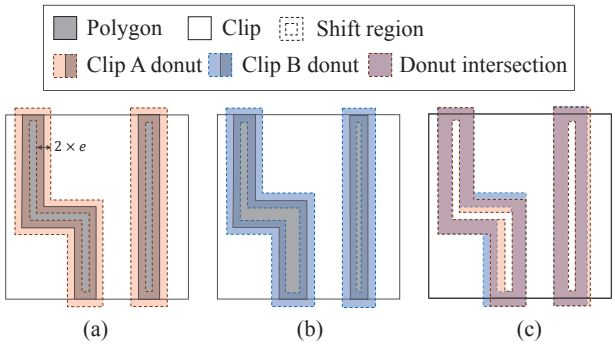

Figure 12: An example of the donut model for edge tolerance consideration. (a)(b) Two given clips. (c) The two clips are matched by ECC since their donuts are continuously overlapped.

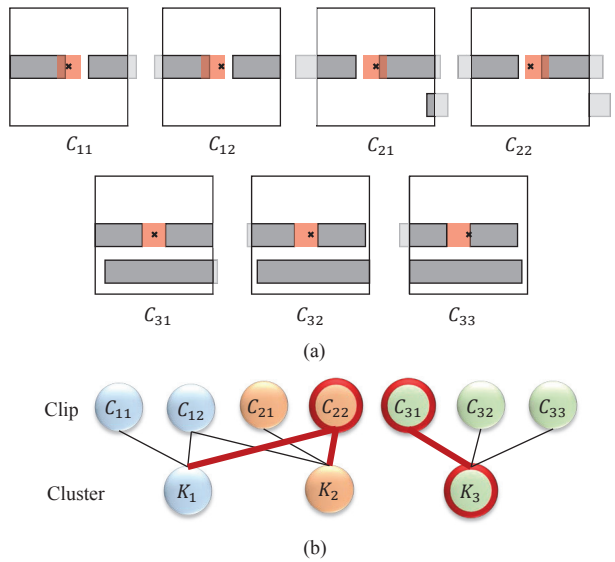

Figure 13: Cluster minimization with set covering. (a) A set of candidate clips including current representative clips and generated clips with shifting under the area/edge constraint. (b) The constructed bipartite graph and the final clustering solution derived by finding a minimum set cover.

pairs of candidate clips can be clustered under the area/edge constraint. For ACC, we use Equation (1) to determine whether the area difference between two clips is smaller than the area matching constraint parameter a. For ECC, we use a donut model to determine whether two clips can be matched with limited edge shifting amount e. Figs. 12(a) and (b) show an example with two clips. A "donut" is created for each pattern by considering the maximum inward and outward edge shifting amount e, which indicates the legal shifting region of the pattern edges. If the donut regions of two clips are continuously overlapped, as an example shown in Fig. 12(c), the two clips are regarded as matched under the edge constraint.

3.4 Cluster Number Minimization with Set Covering

Two current clusters from exact clip matching may be further merged if their shifted clip candidates are matched with area/edge tolerance. Fig. 13(a) shows an example, where C_{11} and C_{21} are two representative clips of two current clusters K_1 and K_2 respectively centered on the marker centers. By shifting C_{11} rightward and shifting C_{21} leftward, two candidate clips C_{12} and C_{22} are generated. Since C_{12} and C_{22} are matched, either of them can be the representative clip of the two clusters, leading to the reduction in the cluster number by 1.

To optimally select a new set of representative clips to minimize the cluster number, we transform the problem into a well-known set cover problem. A lookup table is first constructed to record the matching relationships among all candidate clips including current representative clips and generated clips with shifting under the area/edge constraint. According to the lookup table, we construct a bipartite graph $G = (V_K, V_C, E)$, where each vertex $v_{k_h} \in V_K$ denotes the h-th current cluster K_h, $v_{c_{ij}} \in V_C$ represents the j-th candidate clip C_{ij} of K_i, and an edge $e(v_{c_{ij}}, v_{k_h}) \in E$ indicates that C_{ij} can be a representative clip for K_h. Then, the cluster number can be minimized by selecting a set of $V'_C \subseteq V_C$ with the minimum $|V'_C|$ such that every $v_{k_h} \in V_K$ is connected to some $v_{c_{ij}} \in V'_C$. By regarding V_K as a set of elements and regarding each $v_{c_{ij}} \in V_C$ as an element subset, the optimization problem we solve is equivalent to the conventional set cover problem.

Fig. 13 demonstrates an example of cluster number minimization using set covering. Fig. 13(a) gives seven clip candidates, where C_{i1} are the representative clips of current clusters from exact clip matching, and C_{ij}, $j > 1$, are the generated candidates shifted from C_{i1}. Note that we do not list all shifted candidates due to space limit. Fig. 13(b) is the constructed bipartite graph, where any $v_{c_{ij}}$ is connected to v_{k_i} since any clip candidate generated from C_{i1} can represent the cluster K_i. For example, $v_{c_{11}}$ and $v_{c_{12}}$ are connected to v_{k_1}. In addition, if $C_{i_1j_1}$ and $C_{i_2j_2}$ are matched, the edges $e(v_{c_{i_1j_2}}, v_{k_{i_2}})$ and $e(v_{c_{i_2j_2}}, v_{k_{i_1}})$ will be added into the bipartite graph. For example, $v_{c_{12}}$ is connected to v_{k_2} and $v_{c_{22}}$ is connected to v_{k_1} because C_{12} and C_{22} are matched. To minimize the cluster number, $v_{c_{22}}$ and $v_{c_{31}}$ are chosen to cover all the cluster vertices below, and the clusters covered by the same clip will be merged into one cluster. Note that C_{22} and C_{31} will be the representative clips of the resulting two clusters.

Since the set covering problem is a well-known NP-complete problem, we may uses a common greedy strategy to select candidate clips: choose the one that can cover the most uncovered clusters at each selection moment. This greedy method can find an optimal solution in the example in Fig. 13. To guarantee an optimal solution can always be found, we also implement an ILP formulation to solve the set cover problem. We show the trade-off between the efficiency and the effectiveness of the two methods in our experimental results.

4 EXPERIMENTAL RESULTS

We implemented our hotspot pattern classification framework in the C++ programming language and conducted experiments on a 2.3GHz machine with 64GB memory. We used CPLEX as the ILP solver [13]. The benchmarks are provided by 2016 CAD contest [14] and the detail information are shown in Table 1. We do not use the scoring mechanism in the contest because in addition to the cluster number, the scoring function also considers the jaccard index (similarity coefficient) between reference results and the results of competition teams. Since the reference results do not guarantee the minimum cluster numbers with clip shifting, the scoring mechanism is not relative to the optimization objective of

Table 1: Benchmark for 2016 CAD Contest.

Benchmark	Case1	Case2	Case3	Case4
#Polygon	77	845	9779	147764
#Marker	16	200	5068	264824
Clip width	200	200	200	250
Clip height	200	200	200	250

Table 2: Hotspot pattern classification comparison with CAD contest

Circuit	Parameter	CAD Contest								Ours			
		Reference		Top1		Top2		Top3		Greedy		ILP	
		#C	CPU(s)	#C	CPU(s)	#C	CPU(s)	#C	CPU(s)	#C	CPU(s)	#C	CPU(s)
Case3	default	70	1.314	68	1.361	85	0.06	85	0.5	55	0.22	55	0.27
	a=0.85	26	1.232	22	3.83	52	0.07	24	0.4	12	0.37	8	0.42
	e=8	52	1.311	54	2.81	74	0.07	72	0.4	35	1.34	34	1.31
Case4	default	72	5.279	410	185	72	21	116	381	59	136	59	133
	a=0.99	31	4.74	25	268	69	21	27	380	18	152	17	154
	e=2	57	4.346	25	267	56	20	101	389	48	169	44	172
compare		1		1.63		1.47		1.3		0.7		0.65	

Table 3: Classification results of our algorithm flow with and without clip shifting.

Circuit	Parameter	Reference		Ours with Clip Shifting				Ours Without Clip Shifting			
				Greedy		ILP		Greedy		ILP	
		#C	CPU(s)	#C	CPU(s)	#C	CPU(s)	#C	CPU(s)	#C	CPU(s)
Case1	default	8	0.903	8	0.01	8	0.02	8	0.02	8	0.02
	a=0.95	4	1.808	3	0.01	3	0.02	3	0.02	3	0.02
	e=4	5	1.324	5	0.03	5	0.05	5	0.02	5	0.03
Case2	default	26	1.167	20	0.03	20	0.05	26	0.03	26	0.04
	a=0.9	10	1.168	5	0.06	4	0.1	8	0.04	7	0.05
	e=4	18	0.874	13	0.2	13	0.24	18	0.05	18	0.07
Case3	default	70	1.314	55	0.22	55	0.27	70	0.22	70	0.24
	a=0.85	26	1.232	12	0.37	8	0.42	15	0.22	13	0.24
	e=8	52	1.311	35	1.34	34	1.31	51	0.26	51	0.31
Case4	default	72	5.279	59	136	59	133	72	134	72	139
	a=0.99	31	4.74	18	152	17	154	26	137	24	135
	e=2	57	4.346	48	169	44	172	64	136	64	135
compare		1		0.74		0.71		0.92		0.90	

the pattern classification problem. Therefore, we only compare the cluster numbers in our experiments. Note that since the contest did not verify the correctness of pattern clustering, the results from each team may not follow the area and edge constraints.

Table 2 lists the results compared with those in 2016 CAD Contest at ICCAD, which only releases the results of Case 3 and Case 4. In the table, "default" in the column "Parameter" means that no area and edge tolerance is given, and thus only exactly identical clips will be in the same cluster. a and e are respectively the area and edge constraint parameters. It can be observed from the table that our classification framework derives much better results compared to the top three teams. Compared to the reference results, our framework can averagely reduces the cluster numbers by 30% and 35% with the greedy and the ILP-based methods respectively.

We also compare the pattern classification results that with and without adopting clip shifting, which are shown in Table 3. Experiment results show that without clip shifting, our flow reduces the cluster numbers(#C) by about 10% compared to the reference results. Considering clip shifting can further improve the cluster numbers by about 20%. The runtime difference between the ILP-based and the greedy-based algorithms is not obvious. It may be because the cluster number after exact clip matching is not huge for the given testcases, and thus the problem size of set covering is also kept small. Since the ILP-based method may become computationally expensive as the problem size increases, the greedy strategy for set covering could be adopted to efficiently generate good clustering results.

5 CONCLUSION

In this paper, we present a pattern classification framework with cluster number minimization for hotspot detection. Considering the pattern edge or area constraints, our framework classifies similar clips into clusters. A systematic approach is proposed to generate more useful candidate clips to further reduce the cluster number with clip shifting. Finally, the optimal clustering result can be derived by solving a set cover problem. Experimental results demonstrate the effectiveness of our algorithm flow.

REFERENCES

[1] D. Ding, A. J. Torres, F. G. Pikus, and D. Z. Pan, "High performance lithographic hotspot detection using hierarchically refined machine learning," *Proc. ASP-DAC*, pp. 775–780, 2011.

[2] D. G. Drmanac, F. Liu, and L. C. Wang, "Predicting variability in nanoscale lithography processes," *Proc. DAC*, pp. 545–550, 2009.

[3] J. Guo, F. Yang, S. Sinha, C. Chiang, and X. Zeng, "Improved tangent space based distance metric for accurate lithographic hotspot classification," *Proc. DAC*, pp. 1169–1174, 2012.

[4] W. Hoppe, T. Roessler, and J. A. Torres, "Beyond rule-based physical verification," *Proc. SPIE*, vol. 6349, pp. 63494X, 2006.

[5] J. Kim and M. Fan, "Hotspot detection on Post-OPC layout using full chip simulation based verification tool: A case study with aerial image simulation," *Proc. SPIE*, vol. 5256, pp. 919–925, 2003.

[6] S.-Y. Lin, J.-Y. Chen, J.-C. Li, W.-y. Wen, and S.-C. Chang, "A novel fuzzy matching model for lithography hotspot detection," *Proc. DAC*, pp. 1–6, 2013.

[7] N. Ma, "Automatic IC hotspot classification and detection using pattern-based clustering," PhD thesis, Engineering lC Mechanical Engineering, University of California, Berkeley, 2008.

[8] K. Madkour, S. Mohamed, D. Tantawy and M. Anis, "Hotspot detection using machine learning," *Proc. ISQED*, pp. 405–409, 2016.

[9] J. Xu, S. Sinha, and C. C. Chiang, "Accurate detection for process-hotspots with vias and incomplete specification," *Proc. ICCAD*, pp. 839–846, 2007.

[10] H. Yao, S. Sinha, C. Chiang, X. Hong and Y. Cai, "Efficient process-hotspot detection using range pattern matching," *Proc. ICCAD*, pp. 625–632, 2006.

[11] Y. T. Yu, Y. C. Chan, S. Sinha, I. H. R. Jiang, and C. Chiang, "Accurate process-hotspot detection using critical design rule extraction," *Proc. DAC*, pp. 1163–1168, 2012.

[12] Y. T. Yu, G. H. Lin, I. H. R. Jiang, and C. Chiang, "Machine-learning-based hotspot detection using topological classification and critical feature extraction," *IEEE TCAD*, vol. 34, no. 3, pp. 460–470, 2015.

[13] IBM ILOG CPLEX Optimizer. http://www-01.ibm.com/software/integration/optimization/cplex-optimizer/

[14] R. O. Topaloglu, "CAD contest in pattern classification for integrated circuit design space analysis and benchmark suite," *Proc. ICCAD*, 2016.